



Local Inference For Fine-Tuned SinLlama Model

Sinhala ASR Text Correction On Fine-Tuned SinLlama (Llama 3 8B) Model CPU Inference / GPU Inference User Guide

Version 1.0 · August 2026 · Local Inference — CPU & GPU Modes

Base Model: SAWithanage/SinLlama-Llama-3-8B-Merged
Fine-Tuning Method: QLoRA using Unsloth and TRL SFTTrainer
Project: Sinhala ASR Transcript Correction

Table of Contents

1. Overview.....	2
2. How the Model Works (General Flow)	2
3. CPU Inference Mode	2
3.1 Installation.....	2
3.2 File Setup	2
3.3 How to Run	2
3.4 Minimum Hardware Requirements.....	3
3.5 Code Walkthrough (local_inference_cpu.py)	3
3.6 Expected Console Output.....	3
4. GPU Inference Mode	4
4.1 Minimum Hardware Requirements.....	4

4.2	Installation.....	4
4.3	File Setup	4
4.4	How to Run	4
4.5	Code Walkthrough (local_inference_gpu.py)	5
4.6	Expected Console Output.....	5
5.	CPU vs. GPU: Side-by-Side Comparison.....	6
5.1	Key Code Differences	6
6.	Summary.....	6

1. Overview

This report explains, in plain terms, how to run a fine-tuned Sinhala speech-to-text correction model on a local computer. The model is based on “SinLlama” (a Llama-3 8B model), which has been customised using LoRA (Low-Rank Adaptation) fine-tuning to fix spelling and grammar errors produced by automatic speech recognition (ASR) systems for the Sinhala language.

Two ways of running the model locally are documented: a CPU-only mode and a GPU-accelerated mode. Both modes load the same base model and the same LoRA adapter files (stored in a folder named “lora_backup”), and both use the exact same instruction prompt and generation logic. They differ mainly in hardware requirements, installation steps, model precision, and performance.

This report is based on the full source code of both scripts (`local_inference_cpu.py` and `local_inference_gpu.py`) together with their accompanying setup documents, so every code reference below reflects what is actually written in the scripts.

2. How the Model Works (General Flow)

Both scripts follow the same four general stages. The only real difference is how the model is loaded onto the hardware (plain full-precision on CPU vs. 4-bit quantized on GPU):

- Load the base model “SinLlama-Llama-3-8B-Merged” from Hugging Face.
- Load the fine-tuned LoRA adapter weights from the local “lora_backup” folder and attach them to the base model.
- Take a piece of noisy/incorrect Sinhala text and wrap it inside a fixed instruction template (the same template used during training).
- Feed the formatted prompt into the model, generate a corrected response, and clean up the output before displaying it.

3. CPU Inference Mode

3.1 Installation

Open a terminal and install the standard Hugging Face libraries needed for CPU execution:

```
pip install torch transformers peft
```

3.2 File Setup

- Place the `local_inference_cpu.py` script and the `lora_backup` folder in the exact same directory.
- The `lora_backup` folder must contain the fine-tuned adapter weights and tokenizer files.

3.3 How to Run

```
cd path\to\your\folder
python local_inference_cpu.py
```

3.4 Minimum Hardware Requirements

Component	Minimum Requirement	Why It's Needed
System RAM	32 GB minimum	Needed to hold the full 16 GB unquantized model, the operating system, and memory spikes that occur during text generation.
CPU	Modern multi-core processor (e.g., Intel Core i7/i9 or AMD Ryzen 7/9)	CPU inference is mathematically heavy; a strong multi-core chip prevents timeouts.
Storage	20 GB free SSD space	Needed to cache/download the model weights.

3.5 Code Walkthrough (local_inference_cpu.py)

Step 1 — Configuration & Loading

Unlike the GPU version, this script does not use any quantization configuration. The base model (SAWithanage/SinLlama-Llama-3-8B-Merged) is loaded directly onto the CPU in full float32 precision:

```
base_model = AutoModelForCausalLM.from_pretrained(
    BASE_MODEL_ID,
    device_map="cpu",
    torch_dtype=torch.float32,
    low_cpu_mem_usage=True
)
```

The tokenizer and the fine-tuned LoRA adapter weights are then loaded from the lora_backup folder and merged onto the base model, exactly as in the GPU script. The model is switched to evaluation mode (model.eval()) to prepare it purely for inference rather than training.

Step 2 — The Inference Function (clean_sinhala_text)

This function is functionally identical to the GPU version, with one key difference: the input tensors are explicitly sent to the CPU instead of the GPU.

```
inputs = tokenizer([formatted_prompt], return_tensors="pt").to("cpu")
```

- Wraps the noisy input inside the same fixed Alpaca-style instruction prompt used during training, instructing the model to fix spelling and grammar while preserving numbers and context.
- Tokenizes the prompt and keeps the resulting tensors on the CPU.
- Calls model.generate() to produce up to 512 new tokens, using a low temperature of 0.1 for consistent, low-randomness corrections.
- Decodes the output, extracts only the text after the “### Response:” marker, and removes leftover special tokens (the end-of-sequence token and “<|eot_id|>”).
- Returns the cleaned Sinhala text, or the string “Error: Could not parse output.” if extraction fails.

Step 3 — Execution & Testing

When run directly, the script defines the same sample noisy Sinhala ASR transcript used in the GPU script, passes it through clean_sinhala_text(), and prints the original noisy text alongside the model's cleaned output.

3.6 Expected Console Output

Based on the print statements in the script, running `local_inference_cpu.py` produces console messages in roughly this order (the exact corrected Sinhala text is produced by the model at run time and can only be seen by actually executing the script with the required RAM and model access):

```

Loading Base Model onto CPU (This may take several minutes and 16GB+ RAM)...
Loading Tokenizer...
Injecting LoRA Adapters...
Model Ready for Inference on CPU!

Original (Noisy) : <the sample noisy Sinhala ASR transcript from the script>
-----
Cleaned Output   : <corrected Sinhala text produced by the model>

```

4. GPU Inference Mode

4.1 Minimum Hardware Requirements

Component	Minimum Requirement	Why It's Needed
GPU VRAM (critical)	8 GB minimum (e.g., RTX 3060, RTX 4060, RTX A2000, or a cloud Tesla T4); 12–16 GB recommended	Holds the compressed, 4-bit quantized model; more VRAM allows longer transcripts without running out of memory.
System RAM	16 GB	Sufficient because the GPU (not system RAM) holds the ~5 GB compressed model; RAM only handles normal OS tasks.
Storage	20 GB free space (preferably SSD)	Needed to download and cache the ~16 GB base SinLlama model weights before compression.
Prerequisite software	NVIDIA CUDA Toolkit installed and configured	Required so Python/PyTorch can communicate with the GPU.

4.2 Installation

Install PyTorch built for CUDA 12.1 (adjust the URL if the machine uses a different CUDA version), then install the remaining libraries, including `bitsandbytes` for 4-bit compression:

```

pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121
pip install transformers peft accelerate bitsandbytes

```

4.3 File Setup

- Place the `local_inference_gpu.py` script and the `lora_backup` folder in the exact same directory.

4.4 How to Run

```

cd path\to\your\folder
python local_inference_gpu.py

```

The first run downloads the ~16 GB base model from Hugging Face and can take several minutes. Subsequent runs load the cached copy from disk directly into the GPU, which is much faster.

4.5 Code Walkthrough (local_inference_gpu.py)

Step 1 — Configuration & Loading

The script first defines a 4-bit quantization configuration (BitsAndBytesConfig) that tells the system to compress the model to 4-bit precision using the “nf4” quantization type, with double quantization enabled and float16 used for computation.

```
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4"
)
```

The base model (SAWithanage/SinLlama-Llama-3-8B-Merged) is then downloaded and loaded directly into GPU memory using this configuration, with device_map="auto" letting the library decide how to place the model on the available hardware. The tokenizer and the fine-tuned LoRA adapters are then loaded from the lora_backup folder and merged onto the quantized base model. Finally, the model is set to evaluation mode (model.eval()), which disables training-only behaviour and prepares it purely for inference.

Step 2 — The Inference Function (clean_sinhala_text)

This function takes one argument: a string of noisy Sinhala text (for example, text produced by an automatic speech recognition system). It performs the following steps:

- Wraps the noisy input inside a fixed “Alpaca-style” instruction prompt — the exact same format used when the model was originally fine-tuned. The instruction tells the model: “You are a Sinhala ASR correction system. Fix the spelling and grammar of the noisy input text while preserving all numbers and context.”
- Tokenizes the formatted prompt and moves the resulting tensors to the GPU (.to("cuda")).
- Calls model.generate() to produce up to 512 new tokens, using a low temperature of 0.1 (which makes the output more deterministic and less random — appropriate for a correction task rather than creative writing).
- Decodes the generated tokens back into text, then extracts only the part after the “#### Response:” marker, and strips out leftover special tokens such as the end-of-sequence token and “<|eot_id|>”.
- Returns the cleaned, corrected Sinhala text. If anything goes wrong during parsing, it returns the string “Error: Could not parse output.” instead of crashing.

Step 3 — Execution & Testing

When the script is run directly, it defines a sample block of noisy Sinhala text (a transcript-like passage containing ASR errors) and passes it through clean_sinhala_text(), printing both the original noisy text and the model's cleaned output for comparison.

4.6 Expected Console Output

Based on the print statements written into the script, running local_inference_gpu.py produces console messages in roughly this order (the exact corrected Sinhala text is generated by the model at run time and can only be seen by actually executing the script on a machine with the required GPU and model access):

```
Configuring 4-bit GPU Quantization...
Downloading and Loading Base Model (SAWithanage/SinLlama-Llama-3-8B-Merged) into VRAM...
Loading Tokenizer...
Injecting LoRA Adapters into Base Model...
✅ Model Successfully Loaded into GPU VRAM!
```

Original (Noisy) : <the sample noisy Sinhala ASR transcript from the script>

 Cleaned Output : <corrected Sinhala text produced by the model>

5. CPU vs. GPU: Side-by-Side Comparison

Aspect	CPU Mode (<code>local_inference_cpu.py</code>)	GPU Mode (<code>local_inference_gpu.py</code>)
Compute device	CPU only (no GPU required)	NVIDIA GPU (CUDA)
Model precision	Unquantized (full precision)	4-bit quantized (NF4, via BitsAndBytes)
Approx. model size	~16 GB (full weights)	~5 GB (compressed 4-bit weights)
System RAM needed	32 GB minimum	16 GB is sufficient
Dedicated VRAM needed	Not applicable	8 GB minimum (12–16 GB recommended)
Extra libraries	torch, transformers, peft	torch, transformers, peft, accelerate, bitsandbytes
Relative speed	Slower (CPU math is heavier)	Faster (GPU-accelerated + compressed model)
Best suited for	Machines without a graphics card	Machines with an NVIDIA GPU

5.1 Key Code Differences

Comparing the two scripts line by line, the actual code differences are small and focused entirely on how the model is loaded and where computation happens:

- The GPU script imports and configures BitsAndBytesConfig for 4-bit quantization; the CPU script does not import or use it at all.
- The GPU script loads the base model with `quantization_config=bnb_config` and `device_map="auto"`; the CPU script loads it with `device_map="cpu"` and `torch_dtype=torch.float32` (full precision, no compression).
- The GPU script sends input tensors to the device with `.to("cuda")`; the CPU script uses `.to("cpu")`.
- Everything else — the LoRA loading step, the Alpaca prompt template, the generation settings (512 max new tokens, temperature 0.1), and the output-cleaning logic — is identical between the two scripts.

6. Summary

- Both modes run the same fine-tuned SinLlama model to correct Sinhala ASR transcription errors.
- CPU mode needs no graphics card but requires much more system RAM (32 GB) and will run more slowly.
- GPU mode is faster and lighter on system RAM (16 GB) because the model is compressed to 4-bit, but it requires an NVIDIA GPU with at least 8 GB of VRAM and the CUDA Toolkit installed.
- In both modes, the `lora_backup` folder (containing the fine-tuned adapter and tokenizer) must sit in the same directory as the script before running it.
